NCollection ERP — Data Platform & Backend Architecture

> **Version**: 1.0 **Date**: July 16, 2026 **Classification**: Internal — Enterprise Engineering Reference **Purpose**: The authoritative design for the system's backbone: the distributed database architecture (database-per-tenant at scale), connection management, backup & point-in-time recovery, replication and sharding, cross-tenant migration orchestration, and the pragmatic service-decomposition ("microservices") roadmap. Every backend and infrastructure task in DELIVERABLE_1_SYSTEM_DESIGN.md implements a piece of this document.
>
> **Audience**: DEV-1 primarily; DEV-2/DEV-3 for the parts their code touches; all AI agents before generating any backend code.

Table of Contents

---

1. Design Goals & Constraints

Goal	Target	Why
Isolation	Zero cross-tenant data paths, provable by automated tests	Multiple client companies' financial data on shared infrastructure
RPO (max data loss)	≤ 1 minute (PITR)	A tenant losing a business day of invoices is an existential trust failure

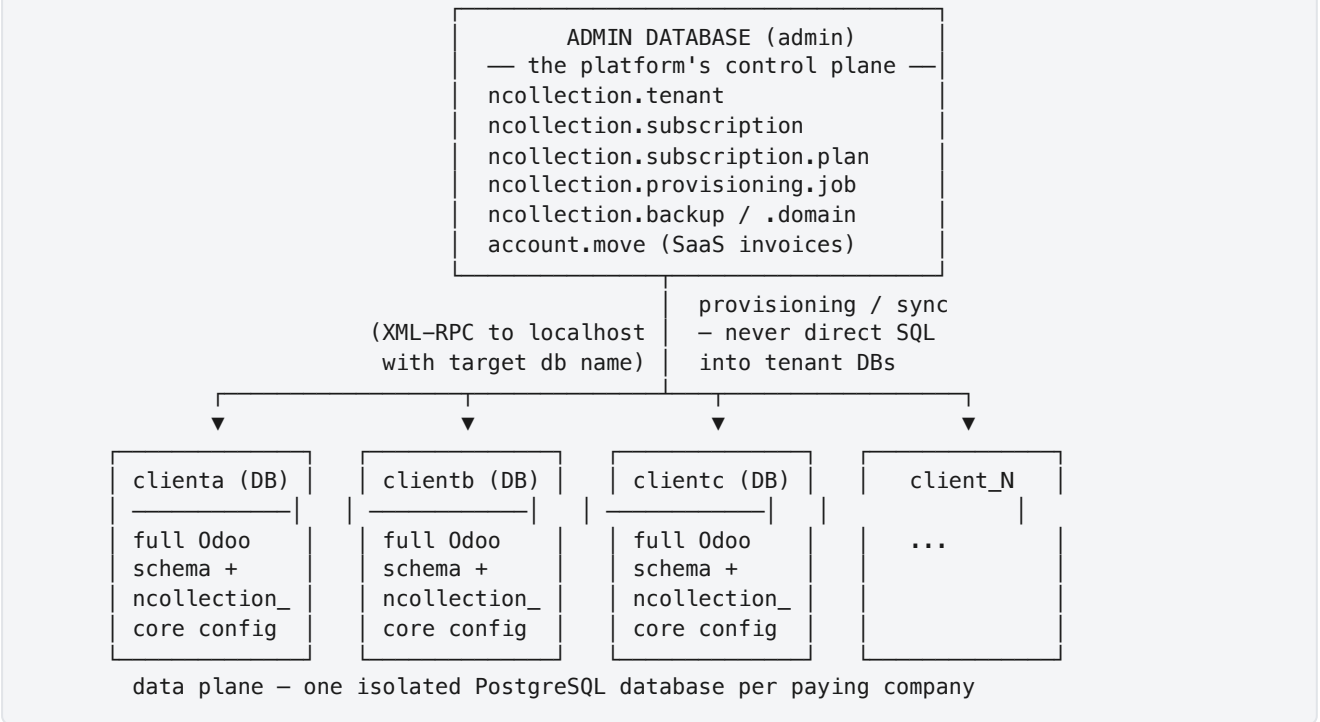
RTO (recovery time)	≤ 1 hour single-tenant restore; ≤ 4 hours full-cluster disaster	SMBs cannot operate without their ERP
Latency	p95 < 500ms for interactive requests; < 200ms for barcode endpoints	ERP is used all day — sluggishness kills adoption
Scale path	1 → 1,000+ tenants without architectural rewrites	The platform must not need re-platforming at success
Operational simplicity	Runnable by a 3-person team	Every component added must earn its operational cost

Constraints that shape everything below:

- Odoo is a **stateful monolith** by design: it opens direct PostgreSQL connections, relies on `LISTEN/NOTIFY` for its realtime bus, stores sessions and attachments on disk, and runs cron per database. We do not fight this; we architect **around** it.
- "Distributed database" here means what it means for every serious Odoo SaaS (including Odoo.com): **many isolated databases, orchestrated as one fleet** — with pooling, replication, PITR, sharding-by-tenant, and region placement. It is *not* a distributed-consensus datastore (no cross-DB transactions exist, and none are needed: a tenant's transaction never spans databases).

---

2. Data Topology Overview



Key invariants (violations are release blockers):

1. **The admin DB is the single source of truth** for who exists, what they pay for, and what they may access. Tenant DBs hold a *projection* of that truth (`ncollection.workspace.config`) written only by the provisioning engine (P2-T01) and the config sync channel (P2-T03).

2. **No tenant DB ever references another database.** No foreign keys, no dblink, no cross-DB queries. The only cross-database actor is the platform control plane, and it speaks XML-RPC through Odoo's public API — never raw SQL into a tenant DB.
3. **A tenant's transaction never spans databases.** This is what makes the fleet horizontally partitionable: any tenant DB can be moved to any cluster at any time (see §6, Stage 3).
4. **Database name == subdomain** (`clienta` ↔ `clienta.ncollectionerp.com`), enforced by `db_filter = ^%d$` . Names are generated once by provisioning (sanitized, collision-checked, reserved-word filtered: `admin` , `www` , `staging` , `api` , `postgres` , `template0/1` are forbidden).

---

3. The Tenant Registry (System Catalog)

The registry is the fleet's metadata brain — it already exists as `ncollection.tenant` and grows fields per phase:

Field group	Fields	Written by	Phase
Identity	`uuid` , `name` , `db_name` , `subdomain`	Provisioning	exists
Lifecycle	`status` (provisioning/active/suspended/archived), onboarding stage	Lifecycle engine	exists / P2-T12
Licensing	plan link, `allowed_module_names` (projection source)	Subscription	P1-T07
Placement	`cluster_id` (which PG cluster hosts this DB), `region`	Provisioning / migration tooling	P10-T03/T05
Operations	`db_size` , `filestore_size` , `last_backup_at` , `backup_status`	Backup + monitoring agents	P2-T05/T10
Domains	domain records, SSL expiry	Domain manager	P2-T06

Why placement lives in the registry: when the platform outgrows one PostgreSQL cluster, routing must answer "which cluster holds `clienta` ?" without guessing. Odoo itself cannot answer this (its `db_host` is static per process), so Stage 3 scaling (§6) runs **one Odoo deployment per DB cluster** and routes at the Nginx layer using a map generated from the registry. Designing the field now costs nothing and prevents a rewrite later.

---

4. Connection Management & Pooling Topology

4.1 The Problem

PostgreSQL connections are expensive (~5–10 MB each). Odoo workers hold pools per database. With database-per-tenant:

```
connections ≈ workers × per-worker pool × active tenant DBs
```

At 4 workers and 30 active tenants you can exhaust the default `max_connections = 100` — the cluster starts refusing logins. Pooling is not optional past ~20 tenants; the topology below is **designed now, deployed in P2-T09**.

4.2 The Topology (CRITICAL — get this exactly right)

```
Odoo HTTP workers (8069)  ──►  PgBouncer (transaction pooling)  ──►  PostgreSQL
Odoo longpolling/bus (8072) ──────────────── DIRECT ──────────────►  PostgreSQL
Odoo cron worker            ──────────────── DIRECT ──────────────►  PostgreSQL
queue_job runner (P2-T01)   ──────────────── DIRECT ──────────────►  PostgreSQL
pgBackRest / backup agent   ──────────────── DIRECT ──────────────►  PostgreSQL
```

Why the split:

- **Transaction pooling** (PgBouncer's high-efficiency mode) hands a server connection to a client only for the duration of a transaction. This breaks any feature needing a **session**: `LISTEN/NOTIFY`, advisory locks, session-level `SET`, prepared statements.
- Odoo's **realtime bus** (chatter, presence, notifications — the longpolling/websocket worker on 8072) is built on `LISTEN/NOTIFY`. Pool it and the bus silently dies: no error, just no realtime updates — a miserable class of bug.
- **Cron and queue workers** hold long transactions (module installs, imports) that would hog pooled server connections and starve HTTP traffic.

PgBouncer configuration principles (P2-T09):

```
[pgbouncer]
pool_mode = transaction
max_client_conn = 2000            ; cheap client slots
default_pool_size = 8             ; server conns per (db, user) pair
reserve_pool_size = 4             ; burst headroom
server_idle_timeout = 300         ; recycle idle server conns
; per-database overrides in [databases] keep one hot tenant
; from starving the fleet:
; clienta = pool_size=12
```

PostgreSQL side: raise `max_connections` modestly (200), and size `shared_buffers` knowing PgBouncer is absorbing the client fan-in. **Monitor pool saturation** (`SHOW POOLS`) from day one — it feeds P2-T10 alerts and later the P8-T04 Prometheus exporter.

4.3 Redis

Redis enters in Phase 2 for **queue backing and caches** (product cache for barcode endpoints P7-T04, dashboard aggregation cache P4-T01). Note honestly: stock Odoo stores **sessions on the filesystem** — that is fine on a single node and solved at Stage 3 (shared filestore / sticky sessions or an OCA session-store module, evaluated then, not before). Do not add infrastructure before its stage needs it.

---

5. Backup, WAL Archiving & Point-in-Time Recovery

5.1 Why `pg_dump` Alone Fails the RPO

A nightly dump means a worst-case **24-hour RPO**: data corrupted at 16:00 loses the whole business day. For a commercial ERP this is unacceptable. The design uses **two complementary layers**:

Layer	Tool	Granularity	RPO	Serves
PITR	pgBackRest (WAL archiving + base backups)	whole cluster, any timestamp	~1 min	Disasters, ransomware, "restore to 15:58"
Logical dumps	`pg_dump --format=custom` per tenant + filestore tar	single tenant, nightly points	24 h	Per-tenant restore, offboarding archives, dev copies

5.2 PITR Design (P2-T04)

```
PostgreSQL ──► archive_command ──► pgBackRest repo ──► S3/Backblaze B2 (encrypted)
                                          │
                weekly full base backup + daily differential
                WAL segments shipped continuously (archive_timeout = 60s)
```

- **Retention**: 2 full sets + WAL to cover 7 days of point-in-time range; monthly fulls kept 6 months.
- **Encryption**: repo-level cipher (`repo1-cipher-type=aes-256-cbc`), key in the secrets store — never in git.
- **Alerting**: WAL archive lag > 5 min pages the team (P2-T10 → P8-T04).
- **The per-tenant nuance** (documented in runbooks, rehearsed in drills): PITR restores the **cluster**, not one database. Single-tenant point-in-time restore = restore the cluster to a scratch instance at timestamp T → `pg_dump` the one tenant DB from it → restore that dump into the live cluster. This is the industry-standard pattern; the runbook scripts it end-to-end so it is a 30-minute procedure, not an incident-time research project.

5.3 Restore Discipline

> **A backup that has never been restored is a hope, not a backup.**

Drill	Frequency	Owner
Automated: verify last-night dumps exist + checksum in B2	Daily	scripted
Restore one random tenant dump to a scratch DB, boot Odoo against it, click through	Monthly	DEV-1
Full PITR restore of the cluster to a scratch instance at an arbitrary timestamp	Quarterly	DEV-1
Complete disaster simulation (fresh VPS from nothing but backups + git)	Before go-live (P3-T13), then annually	team

6. Scaling Stages: From One VPS to Multi-Region

Each stage is triggered by **measured signals**, not by calendar. Skipping ahead adds operational load with no benefit.

Stage 0 — Single node (NOW → ~20 active tenants)

One VPS: Nginx + Odoo (HTTP, cron) + PostgreSQL + Redis. PITR to off-site storage. **Exit signals**: connection warnings, sustained CPU > 60%, backup windows encroaching on business hours.

Stage 1 — Pooling + workload isolation (~20 → ~80 tenants) — *Phase 2 builds this*

Add PgBouncer (topology §4.2) + the **dedicated provisioning/queue runner container** (P2-T01) so heavy jobs never share a process with interactive traffic. Vertically scale the VPS (RAM first — Odoo workers and `shared_buffers` are the pressure points). **Exit signals**: p95 latency creeping under normal load, pool saturation alerts, single-node RAM ceiling reached.

Stage 2 — Read offloading + standby (~80 → ~200 tenants)

PostgreSQL **streaming replication** to a hot standby (P10-T01):

- backups and PITR base-backups run **against the standby** (production stops paying the backup I/O tax),
- heavy analytics (P4 aggregation engine, MIS reports) *may* target the standby via a read-only connection alias where staleness of seconds is acceptable,
- the standby doubles as the failover target (Patroni-automated in P10-T02).

Stage 3 — Sharding by tenant (~200+ tenants) — *P10-T03*

The database-per-tenant model makes this the **cheapest possible sharding**: a tenant is a self-contained unit.

```
Nginx (subdomain → cluster map, generated from the tenant registry)
   ├──► Odoo deployment A ──► PgBouncer A ──► PostgreSQL cluster A (tenants 1–150)
   └──► Odoo deployment B ──► PgBouncer B ──► PostgreSQL cluster B (tenants 151–300)
```

- One Odoo deployment per cluster (Odoo's `db_host` is static per process — this is the constraint that shapes the design).
- The registry's `cluster_id` (§3) is authoritative; an Nginx `map` file is regenerated from it on tenant placement changes.
- **Tenant migration between clusters** = dump → restore → registry update → Nginx map reload → old copy archived. Minutes of downtime for that one tenant, zero for everyone else.

Stage 4 — Multi-region (P10-T05)

Repeat Stage 3 per region (UAE region for PDPL data residency). Region is a placement attribute in the registry; provisioning honors it; backups stay region-local; geo-DNS routes users. The admin control plane remains single-region (it holds no tenant business data — only platform metadata) with a cross-region standby.

---

7. Cross-Tenant Migration Orchestration

Schema/module upgrades must reach **every tenant DB** — the classic fleet problem. The orchestrator (grows inside `ncollection_saas` from Phase 2's runner):

1. **Inventory** — registry lists target DBs + current module versions.
2. **Canary** — upgrade an internal demo tenant + 2–3 volunteer tenants first; soak 24–48 h.
3. **Rolling waves** — batches of N (start N=5) during the off-peak window; per-DB: pre-upgrade `pg_dump` snapshot → `odoo-bin -u <modules> --stop-after-init` → smoke probe (HTTP 200 + login + one ORM read) → mark done in registry.
4. **Failure isolation** — a failed DB is restored from its pre-upgrade snapshot, flagged in the registry, and *excluded* from further waves; the wave continues. One broken tenant must never block 99 healthy ones.
5. **Report** — per-wave outcome to Discord + the admin dashboard.

Rule: no schema change ships until the migration has run green on the canary set. This is why the staging environment (P2-T07) carries multiple tenant DBs permanently.

---

8. Filestore Strategy

Odoo stores attachments on disk at `<data_dir>/filestore/<db_name>/` — **physically separated per tenant** (an isolation guarantee) and **invisible to pg_dump** (hence P2-T05 backs it up explicitly).

Stage	Strategy
0–2	Local NVMe volume, per-tenant tar in nightly backups, size tracked in the registry
3+	Shared object storage (S3-compatible) or NFS so multiple Odoo nodes see one filestore; evaluate OCA `attachment_s3` -class modules first (Rule 2)
4	Region-local buckets (PDPL residency)

Enforce per-plan storage quotas from the registry's `filestore_size` metric (surfaced in the admin dashboard, P2-T15).

---

9. Performance Engineering Rules

The rules every backend PR is reviewed against:

1. **ORM first, SQL when measured** — `read_group` / `search_count` before raw SQL; raw SQL requires a comment with the measurement that justified it, and must be parameterized (never f-strings).
2. **No N+1** — batch with `browse(ids)` prefetching; `read_group` over per-record loops. Reviewers check any loop containing `.search()` or field access on a different model.
3. **Index what you filter** — fields used in record rules (`ir.rule` domains run on *every* query), state fields, and date ranges get `index=True`. License enforcement domains (P1-T10) must be index-backed — they execute on every request.
4. **Cache expensive aggregations** — the P4-T01 engine is the single choke point for dashboard queries: `ormcache` or Redis with explicit invalidation on source writes. No widget queries models directly.
5. **Budgets are tested** — dashboard endpoints < 500ms, barcode endpoints < 200ms, license-enforcement overhead < 5ms/request; the load tests (P3-T03) assert them and the baselines live in `docs/`.
6. **PostgreSQL telemetry always on** — `pg_stat_statements` from Phase 3; the weekly ops review looks at the top-10 by total time.
7. **Cron hygiene** — every `ir.cron` sets sensible batch limits and must be idempotent; anything > 30 s of work belongs on the queue runner, not the cron thread.

Baseline `postgresql.conf` targets (tuned with measurements in P3-T02): `shared_buffers` = 25% RAM · `effective_cache_size` = 75% RAM · `work_mem` = 64MB · `maintenance_work_mem` = 512MB · `random_page_cost` = 1.1 (NVMe) · `wal_compression = on` · `archive_timeout = 60`.

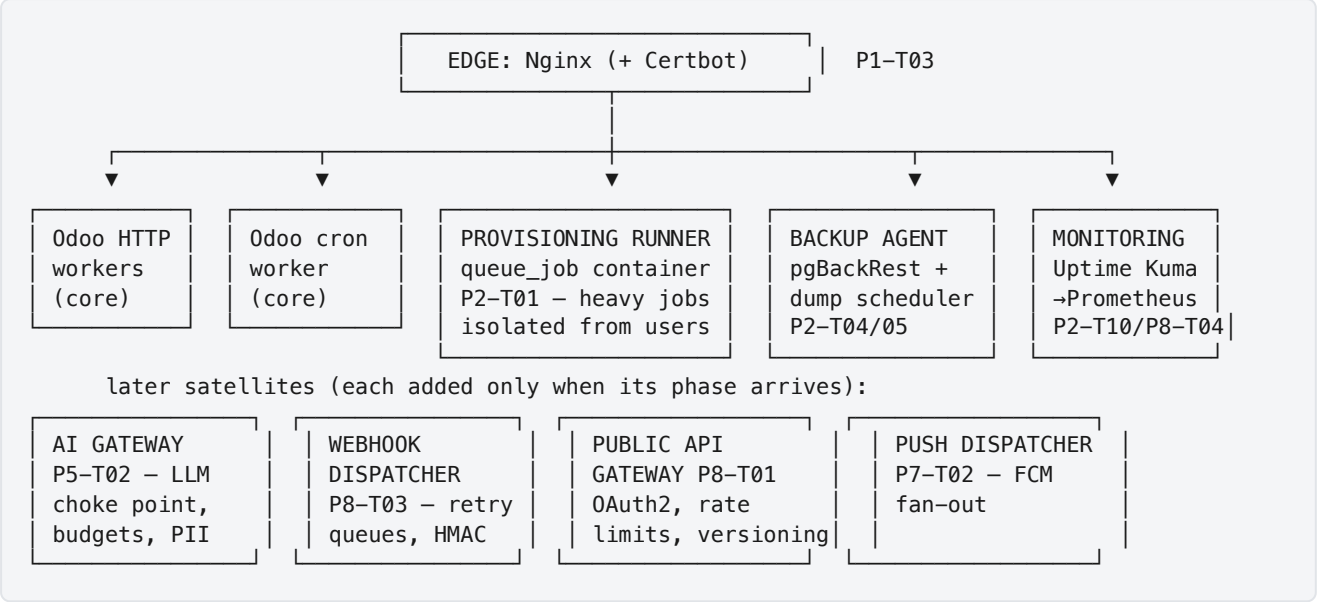
---

10. Service Decomposition Roadmap (Microservices)

10.1 The Honest Position

A 3-developer team should not run 15 microservices — that is how platforms die of operational overload. But a **monolith-plus-satellites** architecture captures the real benefits (fault isolation, independent scaling, security blast-radius reduction) at a cost this team can carry. Odoo remains the core; everything extracted is a **stateless or single-purpose satellite** with one job, its own container, and crash-without-collateral semantics.

10.2 The Decomposition Map



10.3 Extraction Rules

A capability earns its own container **only if** it meets at least two of:

1. **Resource isolation** — heavy/spiky work that would degrade interactive latency (provisioning, backups, AI calls).
2. **Security blast radius** — code talking to the outside world (public API, webhooks, push, LLM providers) that should not share a process with the ORM.
3. **Independent scaling/availability** — needs to scale or fail independently (webhook retries during a tenant's endpoint outage must not consume Odoo workers).

Everything else stays a module inside Odoo. **Explicitly rejected**: splitting ERP domains (sales service, inventory service…) — that path re-implements Odoo badly; the ERP monolith *is* the product's engine.

10.4 Contracts Between Services

- Satellites talk to Odoo through **Odoo's public interfaces only** (XML-RPC/JSON-RPC with dedicated service accounts, or the job queue) — never by importing Odoo internals or writing SQL into tenant DBs.
- Every satellite: health endpoint (`/healthz`), structured JSON logs, restart-safe (idempotent job pickup), resource-limited in compose (`mem_limit` , `cpus`).
- Secrets per satellite (least privilege): the backup agent has DB read creds + B2 write; the AI gateway has LLM keys but **no** DB credentials (it receives context, never fetches it).

---

11. Event & Job Backbone

Phase	Backbone	Used for

2	**OCA `queue_job`** (channels: `provisioning`, `sync`, `mail`)	Async provisioning, config sync, bulk email — DB-backed, transactional with business data, zero new infra
8	+ Redis-backed delivery queues in the webhook dispatcher	Outbound webhook retries/backoff isolated from Odoo
10	Evaluate a real broker (NATS/RabbitMQ) **only if** measured queue latency or fan-out needs demand it	Cross-service events at fleet scale

`queue_job` first is a deliberate choice: jobs enqueue **inside the same DB transaction** as the business change (a subscription activation and its provisioning job commit atomically) — a property most brokers make you build yourself via outbox patterns.

---

12. Capacity Model

Planning numbers (validated against the P3-T03 load-test baseline, then re-measured quarterly):

Per active tenant	Estimate
DB size (SMB, year 1)	0.5–2 GB
Filestore	0.5–5 GB
Concurrent users at peak	3–10
Odoo worker share	~0.15 workers
Pooled PG connections	2–4

Tenants	Infra	Monthly cost (infra)
10	1× CX42 (8 vCPU/16 GB)	~€40 + storage
50	1× CX52 + PgBouncer + runner	~€90
150	2 app nodes + DB node + standby	~€250
500	2 clusters (Stage 3), LB, object storage	~€700

Break-even at a $50/tenant/month average price: infrastructure is < 10% of revenue at every stage — the platform's economics improve with scale, as designed.

---

13. Audit Log Architecture

P8-T05 (task table) covers the OCA `auditlog` integration and field-level tracking. This section covers the *data-platform* side that task doesn't: where audit data lives and how it's queried at scale, since that's a backend-architecture concern, not a module-configuration one.

Storage: per-tenant, not centralized. Each tenant's `auditlog.log` / `auditlog.log.line` records live inside that tenant's own database, exactly like every other table — there is no cross-tenant audit warehouse. This follows directly from §2's isolation model: a centralized audit store would itself become a cross-tenant data-leak surface and defeat the database-per-tenant guarantee. The cost is that "show me all failed-login attempts across all tenants" is not a single query — it's a fan-out the Admin DB has to issue per tenant DB (acceptable at current scale; revisit only if fleet-wide audit querying becomes a frequent operational need, per the extraction rule in §10.3).

What gets audited, and where the line is drawn:

Layer	What's logged	Where
Tenant data layer	Field-level changes on `account.move`, `res.partner`, `sale.order`, `res.users`, all `ncollection.*` models	`auditlog.log` inside the tenant DB (P8-T05)
Application auth	Login success/failure, logout, password reset, IP, user agent	`ncollection.auth.log` inside the tenant DB (P1-T19)
Platform layer	Provisioning actions, subscription state transitions, admin-initiated tenant operations	`ncollection.tenant` chatter + a dedicated `ncollection.platform.audit.log` in the **Admin DB only** — this is platform-operator activity, not tenant data, so it correctly lives outside any tenant boundary
Infrastructure	SSH access, `sudo` use, firewall changes	Host-level `auditd` + centralized log shipping (P2-T08 hardening) — outside the database entirely

Retention & tamper evidence. Per-tenant audit tables inherit that tenant's PITR/backup schedule (§5) — no separate retention mechanism needed. Tamper evidence (hash-chaining consecutive audit rows, flagged as "if feasible" in P8-T05) is scoped per-tenant-DB for the same reason: a chain that spanned tenant boundaries would itself be a cross-tenant coupling. Where UAE PDPL requires a records-of-processing register, that's populated from the platform-layer audit log in the Admin DB, not by querying into tenant DBs.

Query cost. Audit tables are among the fastest-growing in any OLTP schema (every write potentially fans out to N audit rows). Follow the same archival discipline as §8/§9: partition or periodically archive `auditlog.log.line` past the tenant's compliance-required retention window, and never let an unindexed audit-table query run against a live tenant DB during business hours — this is exactly the kind of query the P4-T01 caching lessons apply to if audit data ever needs to power a dashboard (see §14).

---

14. Per-Tenant Cost Dashboard

Gap this section closes: §12's Capacity Model gives platform-wide infrastructure economics (cost per *tier of tenant count*), but nothing today gives a **per-tenant** cost view — neither to NCollection's internal ops (which tenants are expensive relative to what they pay) nor to a tenant Owner (transparent usage-

based billing, if/when the pricing model needs it). This is a real gap, not yet covered by any task in DELIVERABLE_1_SYSTEM_DESIGN.md — flagging it here as the architectural design so a task can be scoped from it (see note at the end of this section).

What "cost" means here — two distinct dashboards, not one:

1. **Internal ops cost dashboard** (Admin DB, NCollection-facing only): per-tenant infrastructure cost attribution — DB size × storage rate, filestore size × storage rate, average worker-time share (derived from §12's "~0.15 workers/tenant" baseline, refined per-tenant from actual request volume), backup/PITR storage share, and (from Phase 5 onward) LLM token spend per tenant if AI features are enabled. Purpose: catch a tenant whose usage pattern makes them unprofitable at their plan tier before it's a quarter-end surprise.
2. **Tenant-facing usage dashboard** (optional, only if a usage-based pricing tier is ever introduced — not assumed by the current flat-plan model in P1-T07): a tenant's own view of their consumption against their plan's included limits (storage, users, API calls, AI tokens if applicable). This is a product decision, not just an architecture one — build it only when a plan tier actually needs it, per the "don't add infrastructure before its stage needs it" principle already applied to Redis in §4.3.

Data sources (both dashboards read from the same underlying signals, aggregated differently):

Signal	Source	Collection method
DB size	`pg_database_size()` per tenant DB	Scheduled probe from the provisioning runner (§10.2), written to the Admin DB's `ncollection.tenant` record — never queried live from a dashboard render
Filestore size	`du` -equivalent per tenant filestore dir (§8)	Same scheduled probe, same cadence
Worker/CPU time	Nginx/Odoo access logs tagged by `db_name` (already present via the routing layer, §2)	Aggregated in the Prometheus pipeline (P8-T04), not queried from Postgres directly
Backup storage	pgBackRest repository size per tenant (§5)	Read from the backup agent's own metadata, same scheduled-probe cadence
LLM token spend (Phase 5+)	AI Gateway's per-tenant token counter (P5-T02, budget enforcement already required there)	Already collected for budget enforcement — this dashboard reuses it, doesn't duplicate it

Why a scheduled probe, not a live query: every one of these signals is expensive to compute live (`pg_database_size` alone is cheap per-call but not at fleet scale run synchronously on every dashboard load) and none of them need sub-hourly freshness for a cost dashboard — this mirrors the §9 rule that dashboard aggregation must be cached with explicit invalidation, not computed per-request. A nightly (or hourly, for the internal ops view) job writes rollup rows into the Admin DB; the dashboard itself only ever reads pre-aggregated data.

Isolation note: the internal ops dashboard lives entirely in the Admin DB and aggregates *metadata about* tenant resource usage — it never queries tenant business data, so it doesn't create a cross-tenant data-access path. The optional tenant-facing dashboard only ever shows that tenant's own numbers, enforced the same way every other tenant-scoped view is (§2, database-per-tenant routing already prevents cross-tenant reads at the connection level).

> **Backlog note**: this section is architecture, not an implementation task yet. If/when built, it fits naturally as a Phase 8 task (alongside P8-T04 Prometheus and P8-T05 Audit Trail, which it depends on for both the metrics pipeline and the audit-log-informed anomaly view) — a candidate task ID would be **P8-T06: Per-Tenant Cost & Usage Dashboard**, dependent on P8-T04 and P5-T02 (for the LLM token signal, if Phase 5 has shipped by then). Not added to the task table in this pass — say the word and it can be formalized there with full acceptance criteria.

> **Document End** Owned by DEV-1. Update when any stage transition happens or any invariant in §2 changes. Changes to §4 (pooling topology) and §5 (PITR) require a PR review by a second developer — these two sections protect the data.